

Operating Systems with Linux

Module 2 – Process Management

Exam Study Notes

Contents

1	Module Overview	3
2	Process Concept	3
2.1	What is a Process?	3
2.2	One-to-One and One-to-Many Program–Process Relationship	3
2.3	CPU-bound vs I/O-bound	3
3	Process Attributes	3
4	Process States	4
4.1	State Transition Diagram	4
4.2	Why Does Each Transition Occur?	4
5	Process Control Block (PCB)	4
5.1	Important PCB Fields	5
5.2	PCB and Context Switching	5
6	Process Scheduling and Scheduling Queues	5
6.1	Scheduling Queues	5
6.2	Three Schedulers	5
6.3	Medium-term Scheduler and Swapping	5
7	Context Switching	6
7.1	Reasons for a Context Switch	6
7.2	Two Basic Steps	6
8	Independent and Cooperating Processes	6
9	CPU Scheduling	6
9.1	Definition	6
9.2	When Can Scheduling Decisions Occur?	6
9.3	Dispatcher	6
10	Scheduling Criteria	7
10.1	Formulas to Memorize	7
11	Preemptive vs Non-preemptive Scheduling	7
12	FCFS Scheduling	7
12.1	Advantages	8
12.2	Disadvantages	8
13	SJF Scheduling	8
13.1	Key Property	8
13.2	Advantages	8
13.3	Disadvantages	8

14 SRTN Scheduling	8
15 Priority Scheduling	8
15.1 Starvation and Aging	9
15.2 Important Relation	9
16 Round Robin Scheduling	9
16.1 Time Quantum: Critical Exam Point	9
16.2 Advantages	9
16.3 Disadvantages	9
17 Multilevel Queue Scheduling	9
18 Multilevel Feedback Queue Scheduling	10
19 CPU Scheduling Algorithms – Master Comparison	10
20 How to Solve CPU Scheduling Numericals	10
21 IPC – Inter-Process Communication	11
21.1 Shared Memory vs Message Passing	11
22 Shared Memory and Producer–Consumer	11
22.1 Producer–Consumer Problem	12
23 Message Passing	12
24 Direct vs Indirect Communication	12
24.1 Symmetric and Asymmetric Direct Communication	12
24.2 Indirect Communication and Mailboxes	13
25 Message Synchronization	13
26 Message Buffering	13
27 Pipes	13
27.1 Pipe Advantages / Disadvantages	13
28 Linux Process Commands	14
28.1 fork()	14
28.2 exec() Family	14
28.3 fork() + exec()	14
28.4 ps	15
28.5 kill	15
28.6 nice and renice	15
29 Special Linux Process Types	16
30 High-Probability 10-Mark Questions	16
30.1 Q1. Explain a Process and its Attributes.	16
30.2 Q2. Explain Process States and State Transition Diagram.	16
30.3 Q3. What is PCB? Explain its Fields and Significance.	16
30.4 Q4. Explain CPU Scheduling and Scheduling Criteria.	17
30.5 Q5. Compare CPU Scheduling Algorithms.	17
30.6 Q6. Explain IPC and Compare Shared Memory and Message Passing.	17
30.7 Q7. Explain Message Passing, Communication Types, Synchronization and Buffering.	17
30.8 Q8. Explain fork(), exec(), ps, kill and nice in Linux.	18

31 5-Mark / Short-Answer Question Bank	18
32 Comparison Questions to Memorize	19
33 Last-Minute Memory Sheet	19
34 Revision Checklist	20

Contents

Module Overview

This module covers process management, CPU scheduling, inter-process communication, and Linux process-management commands. The notes consolidate definitions, mechanisms, diagrams, comparison tables, numerical-solving steps, and university-style question banks from the supplied Module 2 material.

Process Concept

What is a Process?

A **process** is a program under execution, or an executing set of machine instructions.

A process may be:

- a **system process**, executing system code, or
- a **user process**, executing user code.

Program vs Process

Aspect	Program	Process
Nature	Passive entity	Active entity
Meaning	Set of instructions	Program currently executing
Action	Does not initiate execution by itself	Performs the actions specified by the program
Example	Executable file stored on disk	Running instance of that executable

A process contains:

- **Text section:** program code.
- **Data section:** global variables.
- **PCB:** process control block containing information needed to manage the process.

One-to-One and One-to-Many Program–Process Relationship

- **One-to-one:** one instance of a program is running.
- **One-to-many:** multiple instances of one program run simultaneously, or parts of a concurrent program execute concurrently.
- In the one-to-many case, the text section may be the same while the data section is different.

CPU-bound vs I/O-bound

Feature	CPU-bound	I/O-bound
Dominant activity	Computation	I/O operations
Limiting factor	CPU speed	I/O device speed
CPU demand	High	Relatively lower
Typical idea	Long CPU bursts	Frequent I/O activity

Process Attributes

Important attributes maintained by the OS include:

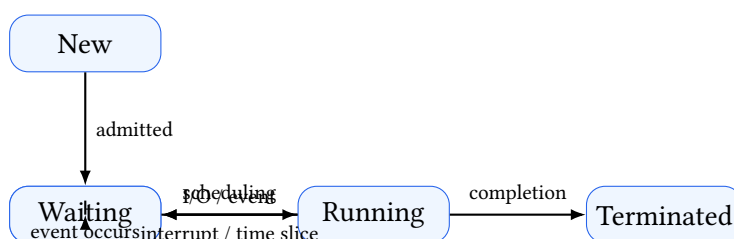
Attribute	Meaning
PID	Unique process identifier.
Program Counter (PC)	Address of the next instruction to execute; important when resuming after a context switch.
Process State	New, ready, running, waiting/blocked, or terminated.
Priority	Determines relative importance for CPU access.
General-purpose registers	Hold data actively used during execution.
Open files	Files currently accessed by the process.
Open I/O devices	Hardware devices currently being used.
CPU scheduling information	Scheduling-related information such as priority and queue pointers.
Memory management information	Information about allocated memory, such as base/limit, page or segment information.
PCB	Data structure containing the process attributes required for management.

Process States

The five core states in the supplied material are:

1. **New**: process is being created.
2. **Ready**: process is ready and waiting for CPU allocation.
3. **Running**: CPU has been allocated and the process is executing.
4. **Waiting / Blocked**: process is waiting for an event such as I/O completion, signal, synchronization, or another required event.
5. **Terminated**: process has completed or has been terminated.

State Transition Diagram



Why Does Each Transition Occur?

Transition	Typical cause
New → Ready	Process is loaded into memory and waits for CPU.
Ready → Running	CPU is allocated to the process.
Running → Ready	Time slice expires or process is preempted.
Running → Waiting	Process requests I/O, waits for a message/event, or requests a resource.
Running → Terminated	Process completes, parent terminates it, or OS terminates it due to an error/resource issue.
Waiting → Ready	Waiting event occurs, such as I/O completion or signal reception.

Process Control Block (PCB)

The OS maintains a process table containing an entry for each process. That entry is the **Process Control Block (PCB)**.

The PCB stores information needed to control and manage a process and indicates its progress.

Important PCB Fields

- **PID:** unique process identifier.
- **Process state:** current state.
- **Parent PID:** PID of the parent process.
- **Child PIDs:** identifiers of child processes.
- **Program counter:** next instruction address.
- **Event information:** event for which a waiting process is blocked.
- **Memory management information:** base/limit, page tables, segment tables, etc.
- **CPU registers:** saved register contents.
- **CPU scheduling information:** priority, scheduling queue pointers, and related information.
- **I/O status:** allocated devices, opened files, current file positions, etc.

PCB and Context Switching

When the CPU moves from one process to another:

1. the kernel saves the old process's context in its PCB;
2. the kernel restores the next process's saved context;
3. execution resumes from the appropriate instruction.

Process Scheduling and Scheduling Queues

The objective of multiprogramming is to keep the CPU busy by ensuring that there is a process ready to execute whenever possible.

When multiple processes compete for the CPU, the OS must decide which process executes next. This is **process scheduling**; the component making the decision is the **scheduler**.

Scheduling Queues

Queue	Where / what it contains	Main purpose
Job queue	Processes entering the system, typically associated with secondary storage	Holds jobs waiting to enter the execution environment.
Ready queue	Ready processes in main memory	Processes waiting for CPU allocation.
Device queue	Separate queue for each I/O device	Processes waiting for a particular I/O device.

The supplied material notes that ready and device queues are generally maintained as linked lists whose nodes contain PCBs.

Three Schedulers

Scheduler	Main job	Effect
Long-term scheduler	Selects jobs to bring into the system / memory	Controls the degree of multiprogramming.
Medium-term scheduler	Swaps processes out of and back into memory	Temporarily reduces/increases the degree of multiprogramming.
Short-term scheduler	Selects a ready process for CPU execution	Runs very frequently and directly influences CPU scheduling.

Medium-term Scheduler and Swapping

The medium-term scheduler, also called the **swapper**, may remove a process from the ready queue or CPU, store it on secondary storage, and later bring it back to memory. This is called **swapping**.

Context Switching

A **context switch** is the mechanism of saving the context of the currently running process and loading/restoring the context of another process.

Reasons for a Context Switch

A context switch may occur when:

- current process terminates;
- current time slice expires;
- process waits for I/O or another resource;
- a higher-priority process becomes ready;
- process voluntarily relinquishes CPU through a system call.

Two Basic Steps

1. **Save context:** kernel saves the current process context into its PCB.
2. **Load/restore context:** kernel loads the context of the next process.

Cost: context switching consumes CPU time and does not perform productive application work during the switch. Therefore excessive switching creates overhead.

Independent and Cooperating Processes

Feature	Independent process	Cooperating process
Effect on other processes	Cannot affect or be affected by another process's execution	Can affect or be affected by another process
Data sharing	No sharing required	Shares data/resources/information
Communication	Not required	Required
Synchronization	Generally not required for cooperation	Coordination/synchronization may be necessary
IPC	Not necessary	Uses IPC mechanisms
Example	Calculator and music player running independently	Word processor and printer process co-operating on a print job

CPU Scheduling

Definition

CPU scheduling is the procedure used to decide which ready process should receive the CPU.

A **scheduling algorithm** is the algorithm used by the scheduler to select the process.

When Can Scheduling Decisions Occur?

The supplied CPU Scheduling material identifies these situations:

1. Running → Waiting.
2. Running → Ready due to an interrupt.
3. Waiting → Ready.
4. Process termination.

Dispatcher

The CPU scheduler selects the process, but the **dispatcher** gives control of the CPU to that selected process.

Dispatcher steps:

1. perform context switching;
2. switch from kernel mode to user mode when executing a user process;
3. transfer control to the appropriate instruction of the selected process.

Dispatch latency is the time required by the dispatcher to suspend one process and resume another. Lower dispatch latency means faster process start.

Scheduling Criteria

The main criteria from the supplied material are:

Criterion	Meaning / desired direction
Fairness	Each process should receive a fair share of CPU time. Maximize.
CPU utilization	Percentage of time CPU is busy. Maximize.
Balanced utilization	Utilization of CPU, I/O, memory and other resources. Maximize.
Throughput	Number of processes completed per unit time. Maximize.
Turnaround time	Exit time – entry time. Minimize.
Waiting time	Turnaround time – burst time in the supplied material. Minimize.
Response time	Time from user request until the system begins responding. Minimize / keep predictable.

Formulas to Memorize

$$\text{Turnaround Time} = \text{Exit Time} - \text{Entry Time}$$

$$\text{Waiting Time} = \text{Turnaround Time} - \text{Burst Time}$$

Preemptive vs Non-preemptive Scheduling

Feature	Non-preemptive	Preemptive
CPU removal	CPU is not forcibly taken before process completion/blocking	CPU may be forcibly taken before completion
Process resumes	Usually after it voluntarily releases CPU	May resume many times
Typical trigger	Process blocks or terminates	Interrupt, time slice expiry, or higher-priority process
Examples in supplied material	FCFS, SJF	SRTN, Priority, Round Robin
Overhead	Generally lower	Generally higher because of more context switches

FCFS Scheduling

First-Come First-Served (FCFS) executes processes in arrival order.

- Non-preemptive.
- Ready queue is FIFO.
- First process entering the ready queue gets CPU first.
- Simple to understand and implement.
- Suitable for batch systems.

Advantages

- Simple implementation.
- No need to access a process from the middle of the queue.
- Well suited to batch systems.

Disadvantages

- Average waiting time is not minimal.
- A long CPU-bound process can make many short I/O-bound processes wait.
- CPU and I/O utilization can suffer in such situations.
- Not suitable for time-sharing systems where processes need equal CPU opportunities.

Exam keyword: Convoy-like effect – a long CPU-bound process at the front can delay many short processes.

SJF Scheduling

Shortest Job First (SJF), also called Shortest Process Next / Shortest Request Next in the supplied material, is **non-preemptive**.

The process requiring the shortest CPU burst among available processes is selected.

Key Property

SJF generally reduces average waiting time because short jobs are executed before long jobs.

Advantages

- Short processes run before long processes.
- Reduces waiting time for short processes.
- The reduction can lower average waiting time.

Disadvantages

- Requires an estimate of CPU burst length in advance.
- User-provided estimates may be inaccurate.
- Long processes can suffer starvation if short processes continue to arrive.

SRTN Scheduling

Shortest Remaining Time Next (SRTN) is the **preemptive version of SJF**.

Instead of comparing total CPU burst, it compares the **remaining CPU burst**.

- If a newly available process has a shorter remaining time than the currently running process, the current process may be preempted.
- A long process can be repeatedly delayed by arriving short processes.
- It also requires knowledge/estimation of CPU burst lengths.

Priority Scheduling

In priority scheduling, every process receives a priority. The highest-priority ready process is selected.

- Can be **preemptive or non-preemptive**.
- If two processes have the same priority, the supplied material uses FCFS order to break the tie.
- Priority may be internally determined by system characteristics or externally specified.
- In the supplied examples, a smaller priority number can represent higher priority. Always read the question carefully.

Starvation and Aging

Starvation: a low-priority process may wait indefinitely if higher-priority processes continually arrive.

Aging: gradually increase the priority of a waiting low-priority process. This eventually allows it to obtain the CPU.

Important Relation

The supplied material notes:

- SJF and SRTN can be viewed as special cases of priority scheduling.
- Priority is inversely related to next CPU burst: shorter burst $\Rightarrow$ higher priority.

Round Robin Scheduling

Round Robin (RR) is a **preemptive** scheduling algorithm commonly associated with time-sharing.

Each ready process receives a fixed CPU interval called a **time quantum** or **time slice**.

1. Ready queue is treated as a circular queue.
2. First process gets one quantum.
3. If it finishes, it exits the queue.
4. If it does not finish, it is preempted and placed at the end of the queue.
5. Next process receives its quantum.
6. Continue until all processes finish.

Time Quantum: Critical Exam Point

- Quantum too small $\rightarrow$ many context switches $\rightarrow$ high overhead.
- Quantum too large $\rightarrow$ RR approaches FCFS.
- The supplied material states that ideally the quantum should allow about 80% of processes to complete within one quantum.

Advantages

- Efficient for time-sharing.
- Improves fairness.

Disadvantages

- Even short processes may take longer overall, reducing throughput.
- Requires timer support for timeouts.

Multilevel Queue Scheduling

Multilevel Queue Scheduling is designed for systems where processes can be divided into groups according to their needs.

Example groups:

- system processes,
- interactive processes,
- batch processes.
- Ready queue is partitioned into separate queues.
- Each queue can have its own scheduling algorithm.
- Example from the supplied material: FCFS for batch processes and RR for interactive processes.
- Processes are permanently assigned to a queue.

- Higher-priority queues are executed before lower-priority queues.

Main weakness: lower-priority queues may suffer starvation if higher-priority queues continuously have work.

Multilevel Feedback Queue Scheduling

Multilevel Feedback Queue (MLFQ), also called multilevel adaptive scheduling in the supplied material, improves on multilevel queue scheduling.

Core difference: processes are **not permanently assigned** to a queue.

A process may move:

- down to a lower-priority queue if it consumes too much CPU time;
- up to a higher-priority queue if it waits too long, helping prevent starvation.

Example arrangement from the supplied material:

- Q1: Round Robin, quantum 5 ms.
- Q2: Round Robin, quantum 10 ms.
- Q3: FCFS.

A process that does not finish in Q1 can be moved to Q2; if it still does not finish, it can be moved to Q3.

CPU Scheduling Algorithms – Master Comparison

Algorithm	Type	Selection idea	Main issue / strength
FCFS	Non-preemptive	Arrival order	Very simple; poor average waiting in some orders.
SJF	Non-preemptive	Shortest CPU burst	Good average waiting; needs burst estimate; long-job starvation possible.
SRTN	Preemptive	Shortest remaining CPU burst	Responsive to short jobs; more preemption; burst estimate needed.
Priority	Either	Highest priority	Important jobs favored; starvation possible; aging helps.
RR	Preemptive	Fixed time quantum, circular queue	Fair and good for time-sharing; quantum choice is critical.
Multilevel Queue	Usually queue-priority driven	Permanently assigned process groups	Different algorithms per queue; lower queues can starve.
MLFQ	Adaptive / feedback based	Queue position changes with behavior	Flexible and can reduce starvation; more complex.

How to Solve CPU Scheduling Numericals

Use this exact workflow.

Step 1 – Read the question carefully

Identify:

- arrival time,
- CPU burst time,
- priority if present,
- time quantum if RR,
- whether scheduling is preemptive/non-preemptive.

Step 2 – Draw the Gantt chart

Do not calculate waiting times before deciding the exact execution order.

Step 3 – Find exit/completion time

For each process, record the time at which its final CPU execution ends.

Step 4 – Calculate turnaround

$$TAT = CT - AT$$

where CT is completion/exit time and AT is arrival/entry time.

Step 5 – Calculate waiting

$$WT = TAT - BT$$

where BT is CPU burst time.

Step 6 – Calculate averages

$$\text{Average } WT = \frac{\sum WT}{n} \quad \text{Average } TAT = \frac{\sum TAT}{n}$$

IPC – Inter-Process Communication

Cooperating processes need mechanisms to exchange data and information. This is **Inter-Process Communication (IPC)**.

The supplied IPC material identifies two basic models:

1. **Shared memory**
2. **Message passing**

Shared Memory vs Message Passing

Feature	Shared memory	Message passing
Basic idea	Processes read/write a shared memory area	Processes send and receive messages
Kernel involvement	Mainly required to set up shared area; subsequent communication need not require kernel intervention	OS/kernel is involved in send/receive
Speed in supplied material	Faster	More time-consuming
Synchronization	Required to avoid simultaneous conflicting access	Communication semantics provide coordination, but blocking/non-blocking choices matter
Same/different machines	Presented primarily as shared area between processes	Can support processes on different computers in a distributed environment

Shared Memory and Producer–Consumer

In shared memory:

1. one process creates a shared memory segment;
2. other processes attach that segment;
3. all communicating processes read/write through the shared area;
4. synchronization is needed so conflicting simultaneous access does not occur.

Producer–Consumer Problem

- **Producer** creates items.
- **Consumer** consumes items.
- They communicate through a shared buffer.

Bounded buffer: fixed size.

- Producer waits if buffer is full.
- Consumer waits if buffer is empty.

Unbounded buffer: no stated size limit.

- Consumer waits when there is nothing to consume.
- Producer need not wait because the buffer is treated as having unlimited capacity.

Message Passing

Message passing uses two basic system calls:

`send()` `receive()`

The sender gives a message to the OS. The OS stores it in a buffer until the receiver invokes `receive()`, after which the message is delivered.

If no message is available when a blocking receiver calls `receive()`, the receiver is blocked until a message arrives.

Message passing can operate between processes on:

- the same computer, or
- different computers connected by a network.

Direct vs Indirect Communication

Feature	Direct communication	Indirect communication
Addressing	Processes address each other using PID	Communication occurs through mailbox/port
Links	One link between each communicating pair in the supplied model	Multiple links may exist, each corresponding to a mailbox
Processes per link	Associated with two processes	A link may be associated with more than two processes
Requirement	Sender knows receiver PID; symmetric form may require both PIDs	Processes share/know a mailbox ID
Example calls	<code>send(PID2, message)</code>	<code>send(X, message)</code>

Symmetric and Asymmetric Direct Communication

Symmetric: both sender and receiver identify each other.

`send(PID2, message)`
`receive(PID1, message)`

Asymmetric: sender identifies receiver; receiver need not know sender beforehand.

```
send(PID2, message)
receive(id, message)
```

The OS can fill *id* with the sender's PID.

Indirect Communication and Mailboxes

A **mailbox** (or port) is a repository for inter-process messages.

Typical operations:

```
send(X, message)    receive(X, message)
```

The supplied material describes a mailbox as having a unique ID. A process knowing that ID can send to it. Ownership and receiving rights can depend on the mailbox implementation described in the material.

Message Synchronization

Messages may be:

Mechanism	Meaning
Blocking send	Sender is blocked until message is received / accepted according to the supplied model.
Blocking receive	Receiver is blocked until a message becomes available.
Non-blocking send	Sender sends and continues without waiting for delivery.
Non-blocking receive	Receiver gets a message if available; otherwise receives NULL according to the supplied model.

Message Buffering

1. **No buffering**: capacity zero; sender must wait until receiver receives.
2. **Bounded buffer**: fixed capacity *m*; sender can continue while space exists, but blocks when full.
3. **Unbounded buffer**: unlimited capacity; sender does not block due to buffer fullness.
4. **Double buffering**: two buffers alternate so that one may be used while the other is full/being emptied.

Pipes

A **pipe** is a standard IPC mechanism for transferring data between processes.

Key points:

- Provides one-way communication between related processes in the supplied material.
- Has a **read end** and a **write end**.
- Data written at the write end can be read from the read end.
- Data is copied through a shared buffer.
- UNIX uses locks and wait queues to synchronize access.
- If a writer cannot write because the pipe is full or locked, it waits; analogous behavior can occur for readers.

Pipe Advantages / Disadvantages

Advantages from supplied material:

- simplicity,
- efficiency,
- reliability,

- flexibility for communication protocols.

Disadvantage to remember: limited capacity can restrict communication.

Linux Process Commands

The supplied command notes focus on `fork()`, `exec()`, `ps`, `kill`, `nice`, and `renice`.

`fork()`

`fork()` creates a new process by duplicating the currently running process.

- Original = parent process.
- New = child process.
- Child gets a new PID.
- Initially parent and child continue from the point after `fork()`.

Return values:

Return value	Receiver	Meaning
> 0	Parent	Child PID
0	Child	This execution is in child
-1	Caller	Creation failed

`exec()` Family

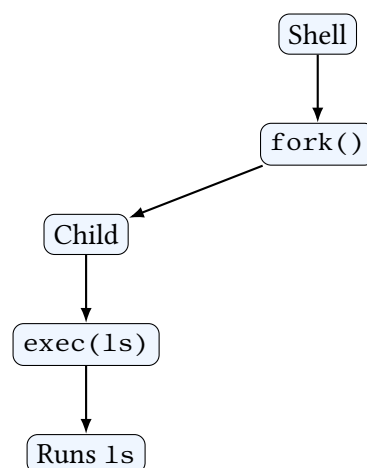
The `exec()` family replaces the current process image with another program.

Critical difference from `fork`: `exec()` does **not** create a new process. The PID remains the same.

Common variants in the supplied material: `execl()`, `execv()`, `execlp()`, `execvp()`, `execle()`, `execve()`.

`fork()` + `exec()`

A common shell pattern is:



	<code>fork()</code>	<code>exec()</code>
Main purpose	Creates a new process	Replaces current process image
PID	New child PID created	Same PID remains
Processes	Parent and child both exist	Old program image is replaced
Memory/program image	Child initially gets copied process state	Current image replaced by new program
Return behavior	Returns differently in parent/child	Returns only on failure in normal <code>exec</code> semantics

ps

ps means Process Status and displays process information.

Important commands from the supplied notes:

- ps: basic process listing.
- ps -e: every process.
- ps -A: all processes; supplied notes equate it with ps -e.
- ps -ef: full details.
- ps -u ;user[: processes belonging to a user.
- ps aux: CPU usage, memory usage, owner, command and related information.
- ps -f: full-format listing.
- ps -l: useful for viewing the NI column.

Important fields:

PID, PPID, UID, CMD, %CPU, %MEM, TTY

kill

Despite its name, kill sends signals to processes.

Signal	Number	Purpose
SIGTERM	15	Graceful termination; default for kill PID in supplied notes
SIGKILL	9	Forcefully kill
SIGSTOP	19	Stop/pause
SIGCONT	18	Continue/resume
SIGHUP	1	Reload configuration in supplied notes
SIGINT	2	Interrupt (Ctrl+C)

Examples:

- kill 2000 - SIGTERM.
- kill -9 2000 - SIGKILL.
- kill -STOP 2000 - stop.
- kill -CONT 2000 - continue.
- pkill firefox - kill processes by name according to the supplied notes.

nice and renice

nice starts a process with a specified scheduling priority.

Nice-value range in the supplied material:

-20 = highest priority, 0 = default, 19 = lowest priority

Therefore:

Lower nice value $\Rightarrow$ higher CPU priority

Examples:

- nice ls
- nice -n 10 firefox
- sudo nice -n -10 firefox
- ps -l - inspect NI column.
- renice 5 -p 2200 - change priority of a running process in the supplied notes.

Special Linux Process Types

From the supplied Linux process notes:

- **Foreground process:** interactive process initiated by the user, normally receiving keyboard input and producing terminal output.
- **Background process:** non-interactive process that can run without direct user input. Example command:

```
sleep 100 &
```

- **Daemon:** background process that starts at system startup and runs continuously, commonly servicing requests.
- **Zombie:** process that has completed execution but still has an entry in the process table.
- **Orphan:** child whose parent has terminated; the supplied material states that the init process adopts it.

High-Probability 10-Mark Questions

These are structured as answer targets rather than random questions.

Q1. Explain a Process and its Attributes.

Answer skeleton

1. Define process.
2. Program vs process.
3. Explain text section, data section and PCB.
4. Explain PID, PC, state, priority, registers, files, I/O devices.
5. Explain scheduling and memory-management information.
6. Mention CPU-bound and I/O-bound if space permits.

Q2. Explain Process States and State Transition Diagram.

Answer skeleton

1. Define process state.
2. Draw five-state diagram.
3. Explain New, Ready, Running, Waiting, Terminated.
4. Explain New→Ready.
5. Explain Ready→Running.
6. Explain Running→Ready.
7. Explain Running→Waiting.
8. Explain Running→Terminated.
9. Explain Waiting→Ready.

Q3. What is PCB? Explain its Fields and Significance.

Answer skeleton

1. Definition.
2. Process table relationship.
3. Draw PCB box.
4. PID, state, parent/child IDs.
5. Program counter.
6. Event information.
7. Memory management.
8. CPU registers.
9. Scheduling information.
10. I/O status.

11. Context-switch significance.

Q4. Explain CPU Scheduling and Scheduling Criteria.

Include:

- definition,
- scheduler,
- when scheduling occurs,
- dispatcher,
- dispatch latency,
- fairness,
- CPU utilization,
- balanced utilization,
- throughput,
- turnaround,
- waiting,
- response time.

Q5. Compare CPU Scheduling Algorithms.

Best table:

- FCFS,
- SJF,
- SRTN,
- Priority,
- Round Robin,
- Multilevel Queue,
- Multilevel Feedback Queue.

For each: type, selection rule, major advantage, major disadvantage, starvation behavior.

Q6. Explain IPC and Compare Shared Memory and Message Passing.

Include:

1. definition of IPC;
2. shared memory model;
3. message passing model;
4. kernel involvement;
5. synchronization;
6. performance distinction stated in the supplied material;
7. same-machine / distributed communication distinction;
8. comparison table.

Q7. Explain Message Passing, Communication Types, Synchronization and Buffering.

Include:

- send/receive,
- direct communication,
- symmetric/asymmetric direct communication,
- indirect communication/mailboxes,
- blocking/non-blocking send,
- blocking/non-blocking receive,
- no/bounded/unbounded/double buffering.

Q8. Explain fork(), exec(), ps, kill and nice in Linux.

Use a table:

Command/system call	Purpose	Must-remember point
fork()	Creates child	Parent gets child PID; child gets 0; failure -1.
exec()	Replaces program image	Does not create a new process; PID remains.
ps	Shows processes	-e, -A, -ef, aux, -u.
kill	Sends signals	SIGTERM 15, SIGKILL 9, STOP 19, CONT 18.
nice	Starts with priority	-20 highest, 19 lowest.
renice	Changes running priority	Changes nice value of an existing process.

5-Mark / Short-Answer Question Bank

1. Define process. How is a process different from a program?
2. Differentiate CPU-bound and I/O-bound processes.
3. List the major process attributes.
4. What is a PCB? Why is it needed?
5. Explain any five PCB fields.
6. What is a process state?
7. Explain Ready and Waiting states.
8. What is process scheduling?
9. Explain job, ready and device queues.
10. Differentiate long-term, medium-term and short-term schedulers.
11. What is context switching?
12. List reasons for context switching.
13. What is dispatch latency?
14. Differentiate independent and cooperating processes.
15. Define CPU scheduling.
16. What is a dispatcher?
17. Define throughput, turnaround time, waiting time and response time.
18. Differentiate preemptive and non-preemptive scheduling.
19. Explain FCFS.
20. Explain SJF.
21. Explain SRTN.
22. Explain priority scheduling.
23. What is aging?
24. Explain Round Robin and time quantum.
25. What happens if RR time quantum is too small or too large?
26. Explain Multilevel Queue Scheduling.
27. Explain Multilevel Feedback Queue.
28. Define IPC.
29. Compare shared memory and message passing.
30. Explain producer-consumer using shared memory.
31. Differentiate bounded and unbounded buffer.
32. Explain direct and indirect communication.
33. Explain blocking and non-blocking communication.
34. Explain pipe IPC.
35. What does fork() do?
36. What are the return values of fork()?

37. Differentiate `fork()` and `exec()`.
38. What is `ps`? Give important options.
39. What is `kill`? Name important signals.
40. Explain nice values and `renice`.

Comparison Questions to Memorize

Program vs Process

Passive vs active; stored instructions vs executing instance.

CPU-bound vs I/O-bound

CPU-heavy vs I/O-heavy.

Independent vs Cooperating

No interdependence/IPC vs interaction/IPC.

Preemptive vs Non-preemptive

CPU can be forcibly taken vs process retains CPU until completion/blocking.

FCFS vs SJF

Arrival order vs shortest burst.

SJF vs SRTN

Non-preemptive total burst vs preemptive remaining burst.

Priority vs Round Robin

Priority-based selection vs equal fixed time slices.

Multilevel Queue vs MLFQ

Permanent queue assignment vs movement between queues.

Shared Memory vs Message Passing

Shared address area vs send/receive messages.

Direct vs Indirect IPC

PID-based addressing vs mailbox/port-based communication.

`fork` vs `exec`

New process vs replacement of current program image.

Last-Minute Memory Sheet

Process

Process = Program under execution

$$\text{Process} = \text{Text} + \text{Data} + \text{PCB}$$

States:

$$\text{New} \rightarrow \text{Ready} \rightarrow \text{Running} \rightarrow \text{Waiting} \rightarrow \text{Ready}$$

and

$$\text{Running} \rightarrow \text{Terminated}$$

Scheduling

$$TAT = CT - AT$$

$$WT = TAT - BT$$

FCFS	Arrival order
SJF	Shortest job
SRTN	Shortest remaining time
Priority	Highest priority
RR	Fixed time quantum
MLQ	Permanent queues
MLFQ	Processes can move between queues

IPC

Shared Memory: read/write shared area

Message Passing: send/receive

Linux

`fork()` = create child

`exec()` = replace program

`ps` = view processes

`kill` = send signal

`nice` = start with priority

Revision Checklist

For revision, make sure the following can be explained without notes:

1. Process definition, program–process relationship, CPU-bound and I/O-bound processes.
2. Process attributes, five process states, state transitions, and PCB fields.
3. Scheduling queues, the three schedulers, context switching, dispatcher, and dispatch latency.
4. Scheduling criteria and formulas for turnaround time and waiting time.
5. FCFS, SJF, SRTN, Priority, Round Robin, Multilevel Queue, and Multilevel Feedback Queue.
6. Gantt-chart procedure for CPU scheduling numericals.
7. IPC, shared memory, message passing, producer–consumer, communication types, synchronization, buffering, and pipes.
8. Linux `fork()`, `exec()`, `ps`, `kill`, `nice`, and `renice`, including important return values, signals, and options.
9. All 5-mark questions and the structured 10-mark answer targets in this document.